

Reinforcement Learning

Class 6

Approximate function evaluation

Mostly based on D. Silver (UCL) lecture 6 from 2015
and chapters 9 and 10 (11 optional reading) from Sutton & Barto's book

Context

- We have covered the core RL methods in the last classes, including the mathematical framework of MDPs, how to reach optimal policies when knowing MDPs and how to handle unknown MDPs to reach optimal policies with TD, MC and Q-learning
... but applicable to scenarios where we can enumerate states (and their value functions as lookup tables)
- This class we will look at approximation methods that allow to address scenarios with many states !

Large-scale Reinforcement Learning

- Reinforcement learning can be used to solve large problems, e.g.
 - Backgammon: 10^{20} states
 - Computer Go: 10^{170} states
 - Helicopter: continuous state space
- So far, we have represented value function by a lookup table
 - Every state s has an entry $V(s)$
 - Or every state-action pair (s, a) has an entry $Q(s, a)$
- Problem with large MDPs:
 - There are **too many states** and/or actions **to store** in memory
 - It is **too slow to learn the value of each state** individually
- How can we scale up the model-free methods for prediction and control from the last two classes?

Value function approximation

- Solution for large MDPs
 - **Estimate value function** with function **approximation** with model with parameters $\mathbf{w}$

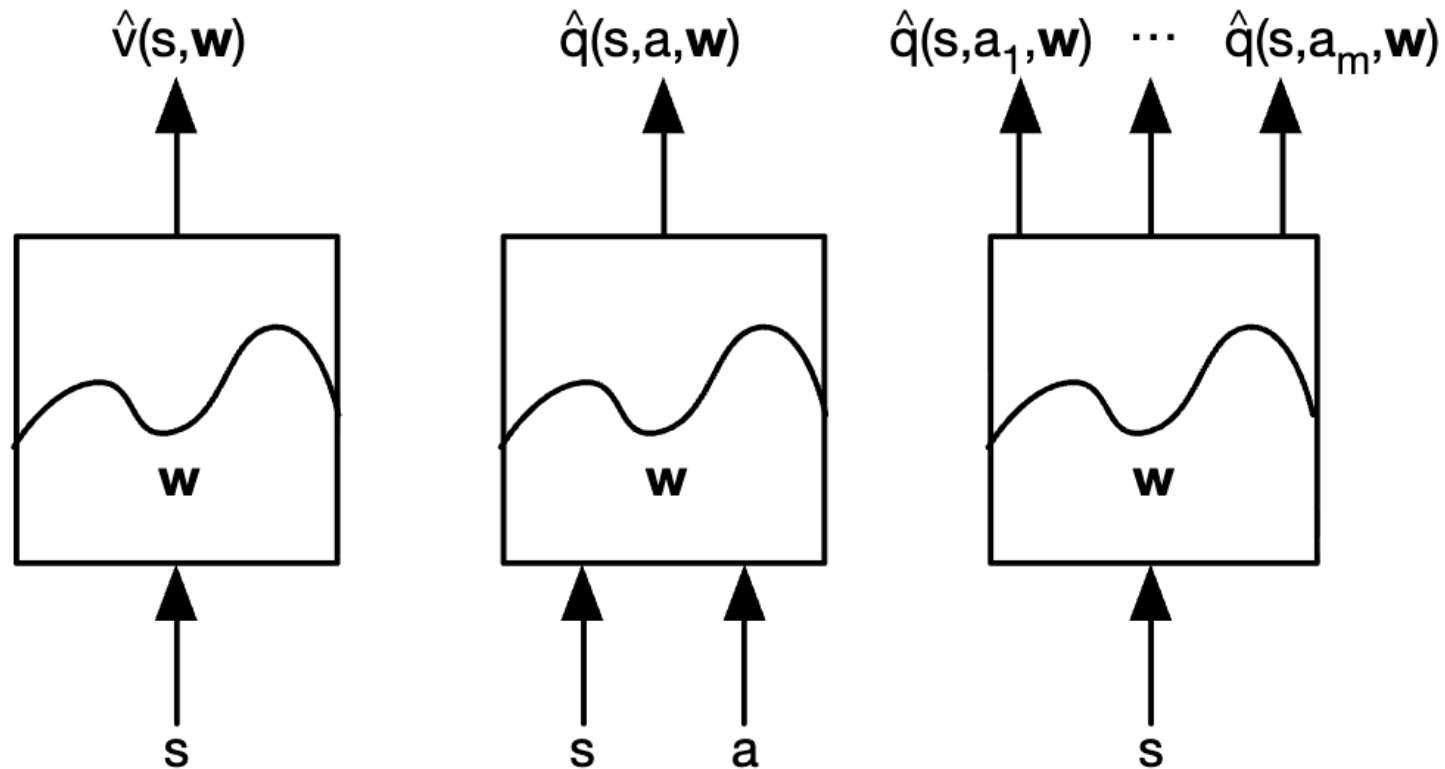
$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$

or $\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$

Prediction task

- **Generalise from seen states to unseen states**; in general $\mathbf{w}$ has less parameters than the number of possible states/ pairs state-action
- Update parameters $\mathbf{w}$ using Monte Carlo or Temporal-Difference learning, seen in previous classes
- Methods come from the field of **supervised learning**

Value function approximation



Different possibilities of mappings (all valid)

Trained models

Value of state s

Action-value of state s performing action a

Action-values of all possible actions from state s

Function approximators

- Different (**supervised**) models can be used as function approximators:
 - Linear models
 - Neural network
 - Decision tree
 - Nearest neighbour
 - ...
- We favour the ones that are **differentiable** as
 - **Linear models**
 - **Neural networks**

and also training methods suitable for **non-stationary**, incremental/ highly **correlated** data typical of RL

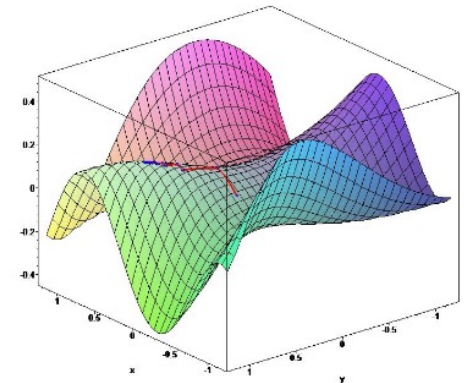
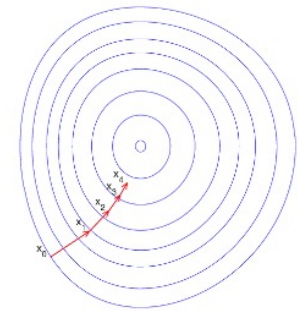
Gradient descent

- $J(\mathbf{w})$ - differentiable function of parameter vector $\mathbf{w}$

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$

- **Aim:** to find the values of $\mathbf{w}$ that reach a **(local) minimum** of J
- Adjust $\mathbf{w}$ in the direction that reduces the gradient (where α is the step size)

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$



Value function approximation with GD

- Goal: find parameter vector $\mathbf{w}$ minimising mean-squared error between approximate value function $\hat{v}(s, \mathbf{w})$ and true value function $v_\pi(s)$ (assumed known for now)

$$J(\mathbf{w}) = \mathbb{E}_\pi [(v_\pi(S) - \hat{v}(S, \mathbf{w}))^2]$$

- Gradient descent finds a **local minimum**

$$\begin{aligned}\Delta \mathbf{w} &= -\frac{1}{2}\alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \mathbb{E}_\pi [(v_\pi(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})]\end{aligned}$$

Value function approximation with Stochastic GD

- **Stochastic gradient descent** samples the gradient (for each exemple)

$$\Delta \mathbf{w} = \alpha (v_{\pi}(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$

- Expected update is equal to full gradient update
- This can work online !
- But remember that normally **we do not know the value function**
!!

Feature vectors

- We typically will represent state by a set of **features** (properties, etc) that allow a learning process

$$\mathbf{x}(S) = \begin{pmatrix} \mathbf{x}_1(S) \\ \vdots \\ \mathbf{x}_n(S) \end{pmatrix}$$

Representing states as features implies changes made for one state will impact many others

- For example:
 - Coordinates in the gridworld
 - Distance of robot from landmarks
 - Trends in the stock market
 - Piece and pawn configurations in chess (or similarly Go, tic tac toe, etc)

Choosing features

- Choosing features appropriate to the task is an important way of adding **prior domain knowledge** to RL systems
- Features should correspond to the aspects of the state space along which **generalization** may be appropriate
- If we add features that combine others, we can make linear models more powerfull considering non-linear interactions
- Non-linear transformations of basic features (polynomials, Fourier transforms, etc) can be used
- Different types of ways to encode multi-dimensional continuous spaces (coarse coding, tile coding, RBF, etc) - check details section 9.5 of Sutton & Barto book

Linear Value Function Approximation

- Represent value function by a linear combination of features
- Objective function is quadratic in parameters $\mathbf{w}$ (MSE)
- Stochastic gradient descent converges on **global optimum**
- Update rule is particularly simple

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^n \mathbf{x}_j(S) \mathbf{w}_j$$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(S) - \mathbf{x}(S)^\top \mathbf{w})^2 \right]$$

$$\nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) = \mathbf{x}(S)$$

$$\Delta \mathbf{w} = \alpha (v_\pi(S) - \hat{v}(S, \mathbf{w})) \mathbf{x}(S)$$

Update = step-size $\times$ prediction error $\times$ feature value

Table lookup features

- Table lookup is a special case of linear value function approximation
- Using table lookup features (states work as binary features)

$$\mathbf{x}^{table}(S) = \begin{pmatrix} \mathbf{1}(S = s_1) \\ \vdots \\ \mathbf{1}(S = s_n) \end{pmatrix}$$

- Parameter vector $\mathbf{w}$ gives value of each individual state working as the look table

$$\hat{v}(S, \mathbf{w}) = \begin{pmatrix} \mathbf{1}(S = s_1) \\ \vdots \\ \mathbf{1}(S = s_n) \end{pmatrix} \cdot \begin{pmatrix} \mathbf{w}_1 \\ \vdots \\ \mathbf{w}_n \end{pmatrix}$$

Incremental Prediction Algorithms

- We have assumed true value function $v_{\pi}(s)$ given by supervisor
- But in RL there is no supervisor, **only rewards**
- In practice, we substitute $v_{\pi}(s)$ for a given target:
 - For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(\mathbf{G}_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

- For TD(0), the target is the TD target (bootstrap estimate)

$$\Delta \mathbf{w} = \alpha(\mathbf{R}_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

- For TD(λ), the target is the λ -return G_t^{λ}

$$\Delta \mathbf{w} = \alpha(\mathbf{G}_t^{\lambda} - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

Monte-Carlo with Value Function Approximation

- Return G_t is an unbiased, noisy sample of true value $v_\pi(S_t)$
- Can therefore apply supervised learning to “training data”
 $\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, \dots, \langle S_T, G_T \rangle$
- For example, using ***linear Monte-Carlo policy evaluation***

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(\textcolor{red}{G}_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(G_t - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)\end{aligned}$$

- Updates typically done after each episode
- Monte-Carlo evaluation converges to a local optimum
- Even when using non-linear value function approximation

Monte-Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):

 Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

 Loop for each step of episode, $t = 0, 1, \dots, T - 1$:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$

Pseudo-code

TD learning with Value Function Approximation

- The TD target is a **biased sample** of true value $v_\pi(S_t)$
- Can still apply supervised learning to “training data”

$$\langle S_1, R_2 + \gamma \hat{v}(S_2, \mathbf{w}) \rangle, \langle S_2, R_3 + \gamma \hat{v}(S_3, \mathbf{w}) \rangle, \dots, \langle S_{T-1}, R_T \rangle$$

- Using **linear TD(0)**
$$\begin{aligned}\Delta \mathbf{w} &= \alpha(R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) \\ &= \alpha \delta \mathbf{x}(S)\end{aligned}$$
- Linear TD(0) typically converges (close) to global optimum, but since $\hat{v}$ appears in the update, this is not a true instance of SGD
- Since we only use GD in the estimate and not the target, they are called *semi-gradient methods*

TD learning with Value Function Approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameter: step size $\alpha > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose $A \sim \pi(\cdot|S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 until S is terminal

Pseudo-code

TD(λ) with Value Function Approximation

- The λ -return G_t^λ is also a biased sample of true value $v_\pi(s)$
- Can again apply supervised learning to “training data”:

$$\langle S_1, G_1^\lambda \rangle, \langle S_2, G_2^\lambda \rangle, \dots, \langle S_{T-1}, G_{T-1}^\lambda \rangle$$

- Forward view linear TD(λ)

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)\end{aligned}$$

- Backward view linear TD(λ)

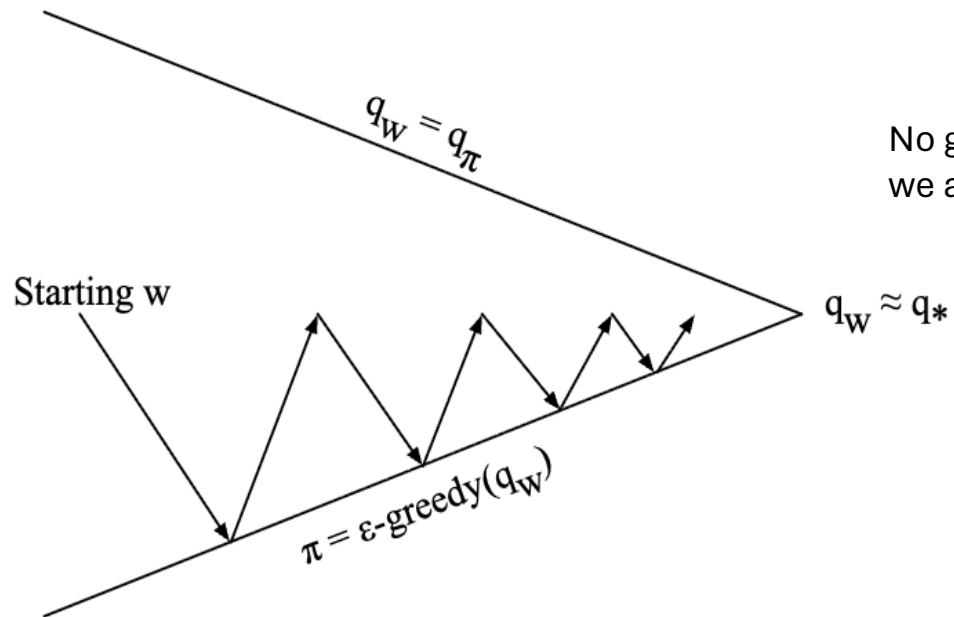
$$\begin{aligned}\delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}) \\ E_t &= \gamma \lambda E_{t-1} + \mathbf{x}(S_t)\end{aligned}$$

Both are equivalent !

$$\Delta \mathbf{w} = \alpha \delta_t E_t$$

Eligibility traces applied
over the features

Control with Value Function Approximation



No guarantees of reaching optimal q since we are using approximations !!

Policy **evaluation** –
Approximate policy evaluation

$$\hat{q}(\cdot, \cdot, \mathbf{w}) \approx q_\pi$$

Policy **improvement** –
 ϵ -greedy Policy improvement

Action-value function approximation

- Approximate the ***action-value function***:

$$\hat{q}(S, A, \mathbf{w}) \approx q_{\pi}(S, A)$$

- Minimise **mean-squared error** between approximate action-value function and true value

$$J(\mathbf{w}) = \mathbb{E}_{\pi} [(q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w}))^2]$$

- Use stochastic gradient descent to find a local minimum

$$-\frac{1}{2} \nabla_{\mathbf{w}} J(\mathbf{w}) = (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha (q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w})$$

Linear Action-Value Function Approximation

- Represent state and action by a feature vector

$$\mathbf{x}(S, A) = \begin{pmatrix} \mathbf{x}_1(S, A) \\ \vdots \\ \mathbf{x}_n(S, A) \end{pmatrix}$$

- Represent action-value fn by linear combination of features

$$\hat{q}(S, A, \mathbf{w}) = \mathbf{x}(S, A)^\top \mathbf{w} = \sum_{j=1}^n \mathbf{x}_j(S, A) \mathbf{w}_j$$

- Stochastic gradient descent update

$$\nabla_{\mathbf{w}} \hat{q}(S, A, \mathbf{w}) = \mathbf{x}(S, A)$$

$$\Delta \mathbf{w} = \alpha (q_\pi(S, A) - \hat{q}(S, A, \mathbf{w})) \mathbf{x}(S, A)$$

Incremental control algorithms

- Like prediction, we must substitute a target for $q_\pi(S, A)$

- For MC, the target is the return G_t $\Delta \mathbf{w} = \alpha(\textcolor{red}{G}_t - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$

- For TD(0), the target is the TD target

$$\Delta \mathbf{w} = \alpha(\textcolor{red}{R}_{t+1} + \gamma \hat{q}(\textcolor{red}{S}_{t+1}, \textcolor{red}{A}_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- For forward-view TD(λ), target is the action-value λ -return

$$\Delta \mathbf{w} = \alpha(\textcolor{red}{q}_t^\lambda - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- For backward-view TD(λ), equivalent update is

$$\delta_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$

$$E_t = \gamma \lambda E_{t-1} + \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

$$\Delta \mathbf{w} = \alpha \delta_t E_t$$

Sarsa with function approximation - algorithm

Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

$S, A \leftarrow$ initial state and action of episode (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 If S' is terminal:

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

 Go to next episode

 Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε -greedy)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})] \nabla \hat{q}(S, A, \mathbf{w})$

$S \leftarrow S'$

$A \leftarrow A'$

Pseudo-code

N-step Sarsa with function approximation - algorithm

Episodic semi-gradient n -step Sarsa for estimating $\hat{q} \approx q_*$ or q_π

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

Input: a policy π (if estimating q_π)

Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$, a positive integer n

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

All store and access operations (S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

Initialize and store $S_0 \neq \text{terminal}$

Select and store an action $A_0 \sim \pi(\cdot | S_0)$ or ε -greedy wrt $\hat{q}(S_0, \cdot, \mathbf{w})$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

 else:

 Select and store $A_{t+1} \sim \pi(\cdot | S_{t+1})$ or ε -greedy wrt $\hat{q}(S_{t+1}, \cdot, \mathbf{w})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

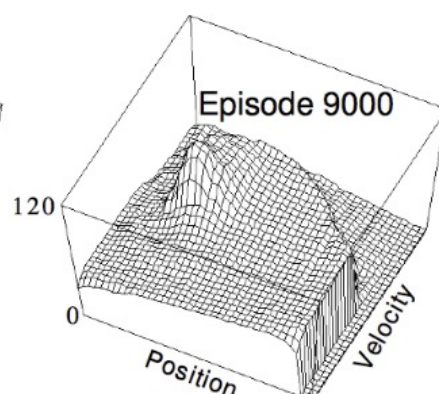
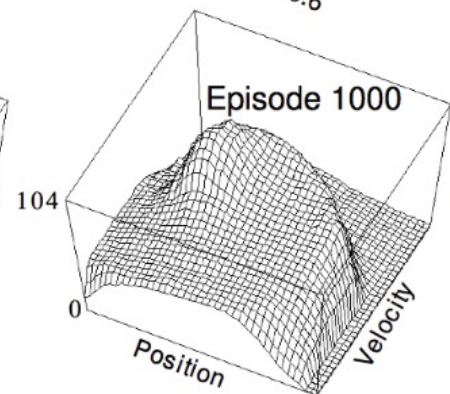
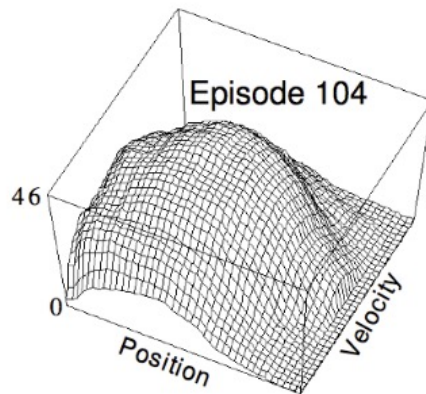
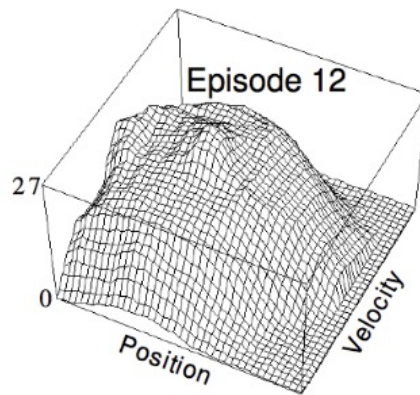
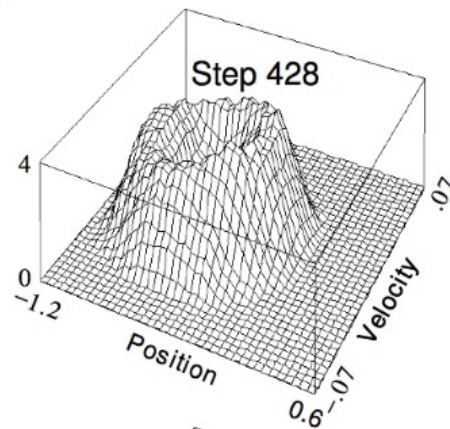
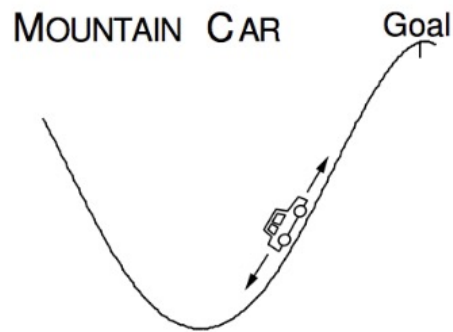
 If $\tau + n < T$, then $G \leftarrow G + \gamma^n \hat{q}(S_{\tau+n}, A_{\tau+n}, \mathbf{w})$ ($G_{\tau:\tau+n}$)

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G - \hat{q}(S_\tau, A_\tau, \mathbf{w})] \nabla \hat{q}(S_\tau, A_\tau, \mathbf{w})$

Until $\tau = T - 1$

Pseudo-code

Example – Mountain Car – Sarsa - linear



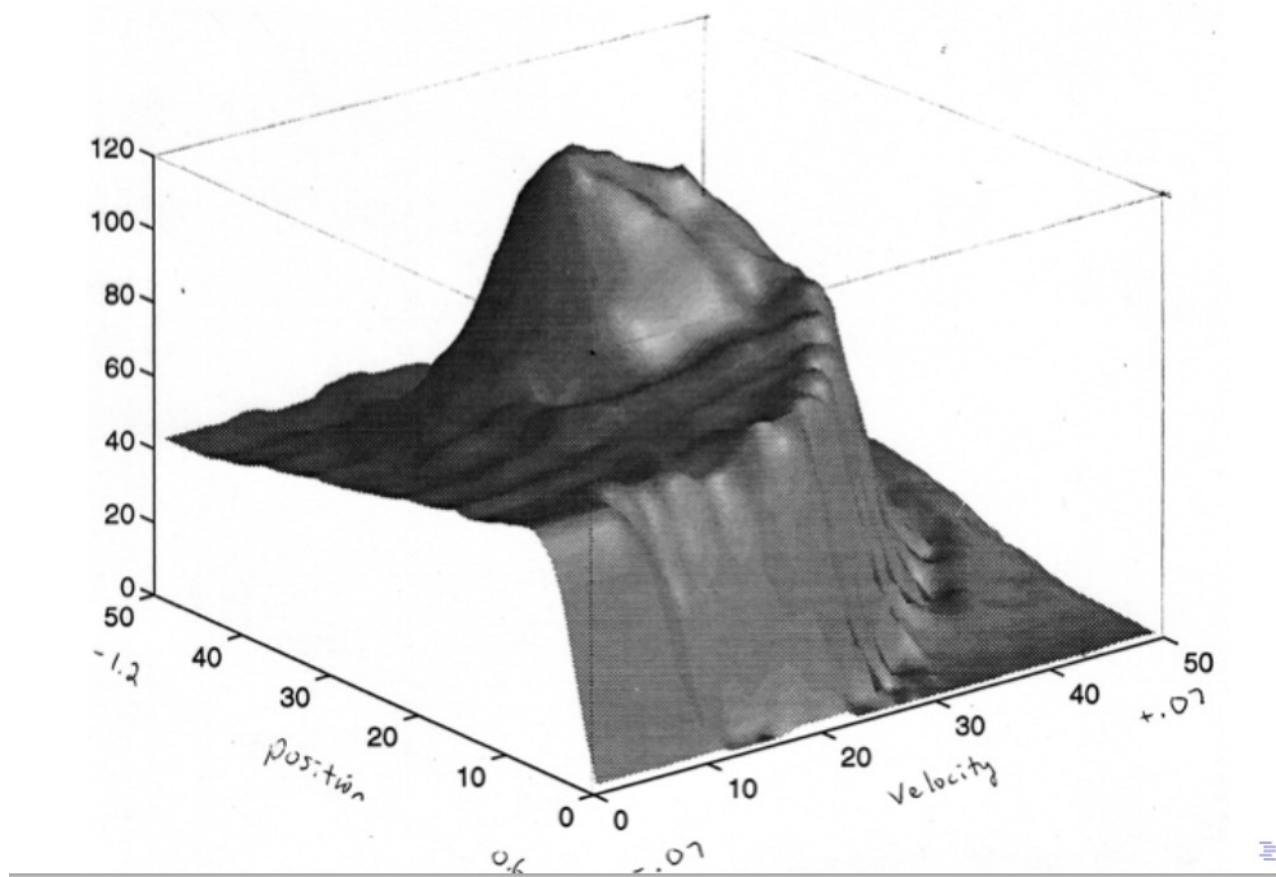
Car needs to go up, but gravity is stronger than the car's engine, and even at full throttle the car cannot accelerate up the steep slope

The only solution is to first move away from the goal and up the opposite slope on the left

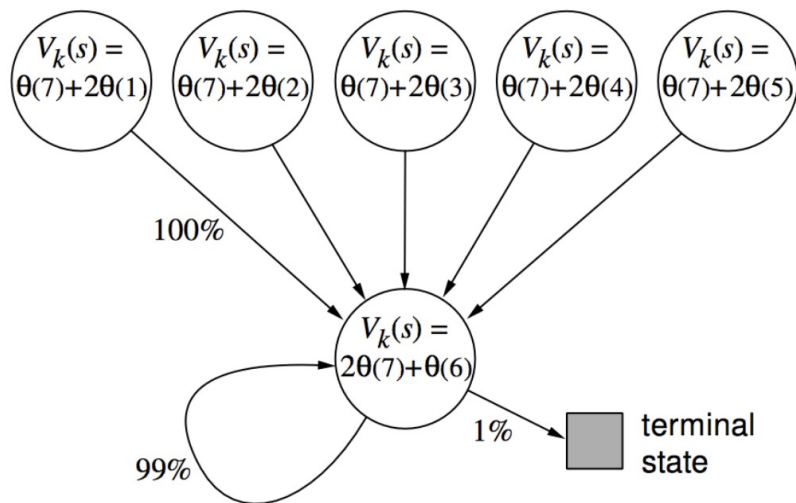
State space – position + velocity

Action: acceleration (binary)

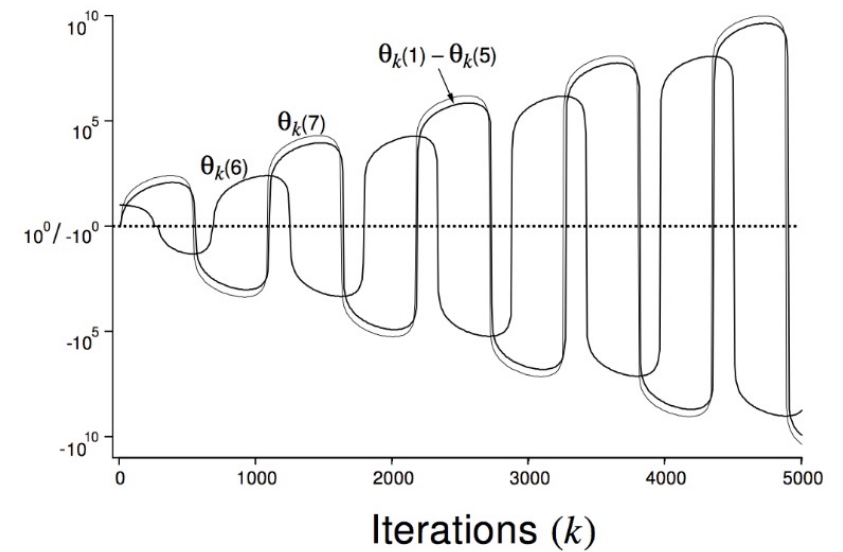
Example – Mountain Car – Sarsa – non linear



Example – when TD learning does not converge



Parameter values, $\theta_k(i)$
(log scale,
broken at ± 1)



Convergence of prediction algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD(0)	✓	✓	✗
	TD(λ)	✓	✓	✗
Off-Policy	MC	✓	✓	✓
	TD(0)	✓	✗	✗
	TD(λ)	✓	✗	✗

X – Means that convergence is not guaranteed (but can work in practice)

Gradient TD learning

- TD does not follow the gradient of any objective function
- This is why TD can diverge when off-policy or using non-linear function approximation
- **Gradient TD** follows true gradient of projected Bellman error

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD	✓	✓	✗
	Gradient TD	✓	✓	✓
Off-Policy	MC	✓	✓	✓
	TD	✓	✗	✗
	Gradient TD	✓	✓	✓

Details on section 11.7 – Sutton & Barto

Convergente of control algorithms

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	✗
Sarsa	✓	(✓)	✗
Q-learning	✓	✗	✗
Gradient Q-learning	✓	✓	✗

(✓) = chatters around near-optimal value function

Batch Reinforcement Learning

- Gradient descent is simple and appealing
- But it is not sample efficient
- Batch methods seek to find the best fitting value function
- Given all the agent's experience (“training data”)

Least squares prediction

- Given value function approximation $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$ and experience $\mathcal{D}$ consisting of $\langle \text{state}, \text{value} \rangle$ pairs

$$\mathcal{D} = \{ \langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle \}$$

- Which parameters $\mathbf{w}$ give the best fitting value function $\hat{v}(s, \mathbf{w})$
- **Least squares algorithm** find parameter vector $\mathbf{w}$ minimising sum-squared errors between $\hat{v}(s, \mathbf{w})$ and target values

$$\begin{aligned} LS(\mathbf{w}) &= \sum_{t=1}^T (v_t^\pi - \hat{v}(s_t, \mathbf{w}))^2 \\ &= \mathbb{E}_{\mathcal{D}} [(v^\pi - \hat{v}(s, \mathbf{w}))^2] \end{aligned}$$

Stochastic Gradient Descent with Experience Replay

- Given experience consisting of <state, value> pairs $\mathcal{D}$

$$\mathcal{D} = \{\langle s_1, v_1^\pi \rangle, \langle s_2, v_2^\pi \rangle, \dots, \langle s_T, v_T^\pi \rangle\}$$

- Repeat:

1. Sample (state, value) from experience

So, basically standard Supervised Learning !!

2. Apply stochastic GD update

$$\langle s, v^\pi \rangle \sim \mathcal{D}$$

$$\Delta \mathbf{w} = \alpha (v^\pi - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

Converges to least squares solution

$$\mathbf{w}^\pi = \underset{\mathbf{w}}{\operatorname{argmin}} LS(\mathbf{w})$$

Experience replay in Deep Q-Networks (DQN)

DQN uses **experience replay** and fixed Q-targets

- Take action a_t according to ϵ -greedy policy
- Store transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in replay memory
- Sample random mini-batch of transitions (s, a, r, s') from D
- Compute Q-learning targets w.r.t. **old**, fixed parameters w^-
- Optimise MSE between Q-network and Q-learning targets

$$\mathcal{L}_i(w_i) = \mathbb{E}_{s,a,r,s' \sim \mathcal{D}_i} \left[\left(r + \gamma \max_{a'} Q(s', a'; w_i^-) - Q(s, a; w_i) \right)^2 \right]$$

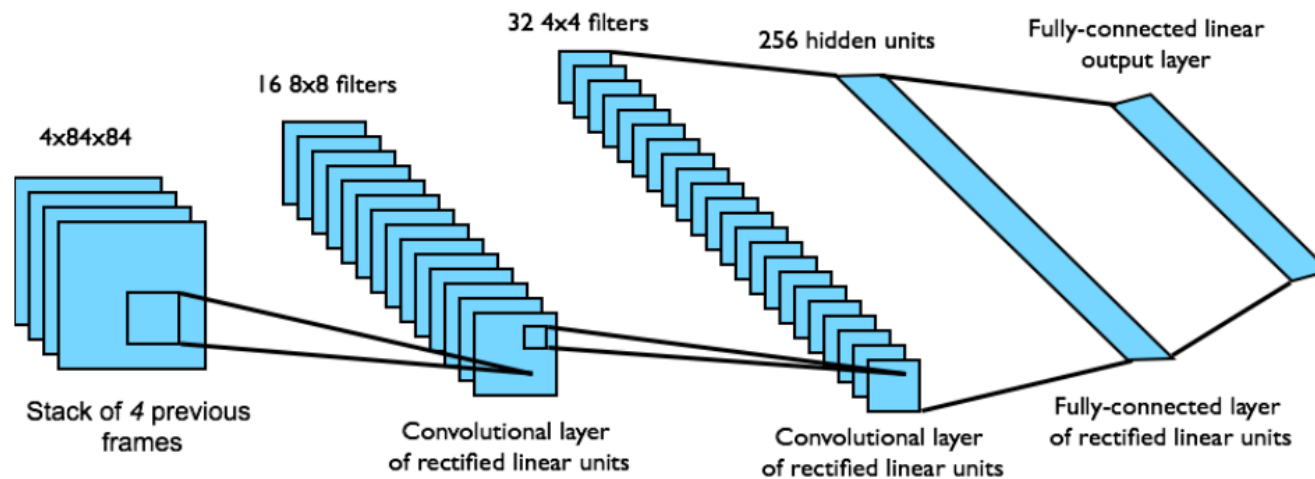
(using a variant of stochastic gradient descent)

<https://arxiv.org/abs/1312.5602>

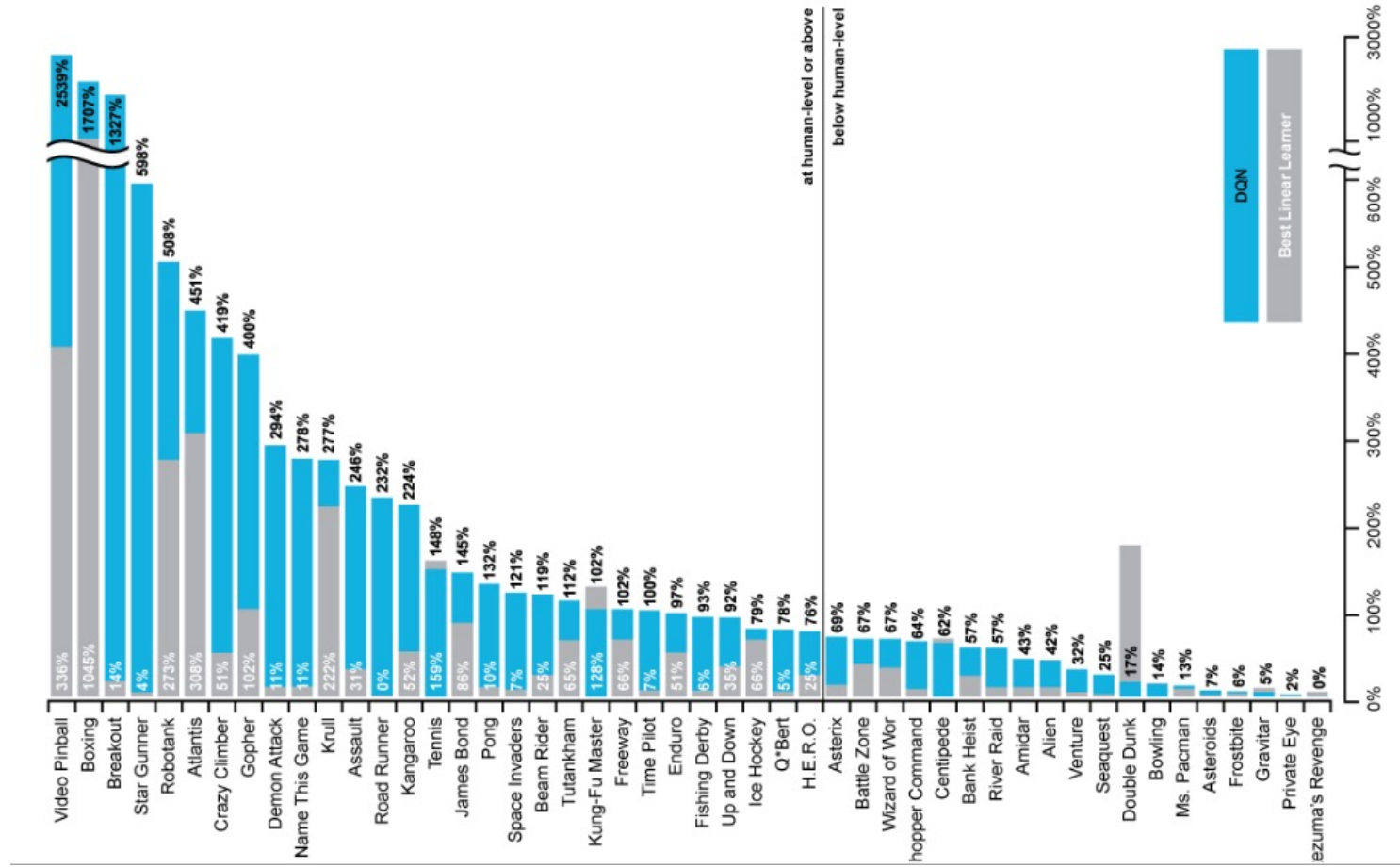
<https://www.nature.com/articles/nature14236>

DQN in Atari

- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is stack of raw pixels from last 4 frames
- Output is $Q(s, a)$ for 18 joystick/ button positions
- Reward is change in score for that step



DQN in Atari - results



DQN in Atari - results

	Replay Fixed-Q	Replay Q-learning	No replay Fixed-Q	No replay Q-learning
Breakout	316.81	240.73	10.16	3.17
Enduro	1006.3	831.25	141.89	29.1
River Raid	7446.62	4102.81	2867.66	1453.02
Seaquest	2894.4	822.55	1003	275.81
Space Invaders	1088.94	826.33	373.22	301.99

Linear Least Squares Prediction

- Experience replay finds least squares solution
- But it **may take many iterations**

- Using linear value function approximation $\hat{v}(s, \mathbf{w}) = \mathbf{x}(s)^\top \mathbf{w}$

we can solve the least squares solution directly by deterministic exact methods (closed form)

Linear Least Squares Prediction

- At minimum of $LS(w)$, the expected update must be zero (gradient is zero)

$$\mathbb{E}_{\mathcal{D}} [\Delta \mathbf{w}] = 0$$

$$\alpha \sum_{t=1}^T \mathbf{x}(s_t)(v_t^{\pi} - \mathbf{x}(s_t)^{\top} \mathbf{w}) = 0$$

$$\sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi} = \sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^{\top} \mathbf{w}$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(s_t)\mathbf{x}(s_t)^{\top} \right)^{-1} \sum_{t=1}^T \mathbf{x}(s_t)v_t^{\pi}$$

- For N features, direct solution time is $O(N^3)$
- Incremental solution time is $O(N^2)$ using Sherman-Morrison method

Linear Least Squares Prediction Algorithms

- But again, we do not know true values v_t^π
- In practice, our “training data” must use noisy or biased samples of v^π

LSMC Least Squares Monte-Carlo uses return

$$v_t^\pi \approx G_t$$

LSTD Least Squares Temporal-Difference uses TD target

$$v_t^\pi \approx R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$$

LSTD(λ) Least Squares TD(λ) uses λ -return

$$v_t^\pi \approx G_t^\lambda$$

Linear Least Squares Prediction Algorithms

LSMC

$$0 = \sum_{t=1}^T \alpha (G_t - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) \mathbf{x}(S_t)^\top \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) G_t$$

LSTD

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t)$$

$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t) (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^\top \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t) R_{t+1}$$

LSTD(λ)

$$0 = \sum_{t=1}^T \alpha \delta_t E_t$$

$$\mathbf{w} = \left(\sum_{t=1}^T E_t (\mathbf{x}(S_t) - \gamma \mathbf{x}(S_{t+1}))^\top \right)^{-1} \sum_{t=1}^T E_t R_{t+1}$$

LSTD algorithm – pseudo-code

LSTD for estimating $\hat{v} = \mathbf{w}^\top \mathbf{x}(\cdot) \approx v_\pi$ ($O(d^2)$ version)

Input: feature representation $\mathbf{x} : \mathcal{S}^+ \rightarrow \mathbb{R}^d$ such that $\mathbf{x}(\text{terminal}) = \mathbf{0}$

Algorithm parameter: small $\varepsilon > 0$

$$\widehat{\mathbf{A}}^{-1} \leftarrow \varepsilon^{-1} \mathbf{I}$$

A $d \times d$ matrix

$$\widehat{\mathbf{b}} \leftarrow \mathbf{0}$$

A d -dimensional vector

Loop for each episode:

Initialize S ; $\mathbf{x} \leftarrow \mathbf{x}(S)$

Loop for each step of episode:

Choose and take action $A \sim \pi(\cdot|S)$, observe R, S' ; $\mathbf{x}' \leftarrow \mathbf{x}(S')$

$$\mathbf{v} \leftarrow \widehat{\mathbf{A}}^{-1 \top} (\mathbf{x} - \gamma \mathbf{x}')$$

$$\widehat{\mathbf{A}}^{-1} \leftarrow \widehat{\mathbf{A}}^{-1} - (\widehat{\mathbf{A}}^{-1} \mathbf{x}) \mathbf{v}^\top / (1 + \mathbf{v}^\top \mathbf{x})$$

$$\widehat{\mathbf{b}} \leftarrow \widehat{\mathbf{b}} + R \mathbf{x}$$

$$\mathbf{w} \leftarrow \widehat{\mathbf{A}}^{-1} \widehat{\mathbf{b}}$$

$$S \leftarrow S'; \mathbf{x} \leftarrow \mathbf{x}'$$

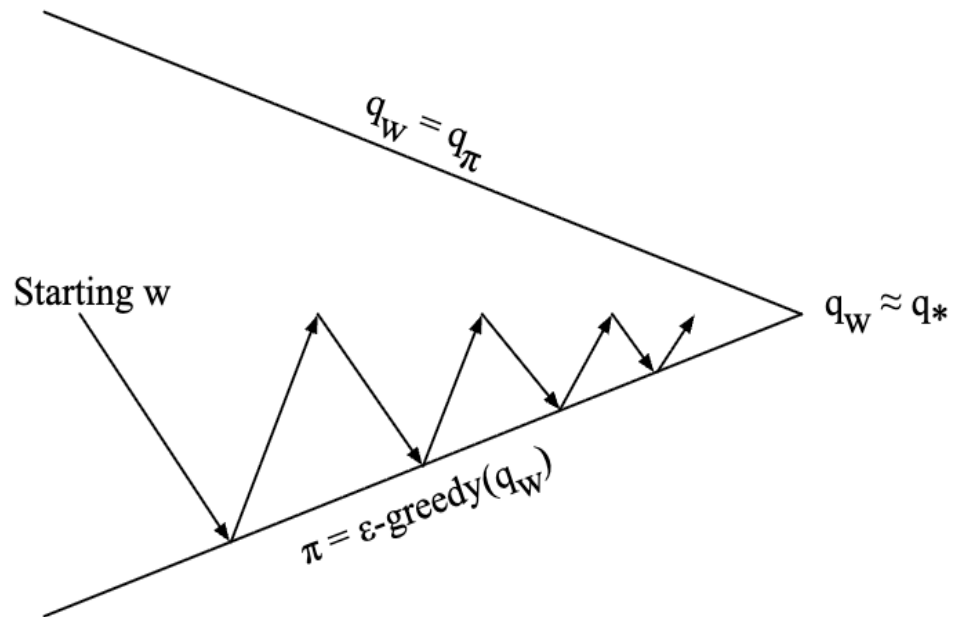
until S' is terminal

Pseudo-code

Convergence of Linear Least Squares Prediction Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✓	✗
	LSTD	✓	✓	-
Off-Policy	MC	✓	✓	✓
	LSMC	✓	✓	-
	TD	✓	✗	✗
	LSTD	✓	✓	-

Least Square Policy Iteration - LSPI



Policy **evaluation** –
Policy evaluation by Least Squares
Q – learning

Policy **improvement** –
Greedy Policy improvement

Least Squares Action-Value Function Approximation

- Approximate action-value function $q_\pi(s, a)$ using linear combination of features $\mathbf{x}(s, a)$ $\hat{q}(s, a, \mathbf{w}) = \mathbf{x}(s, a)^\top \mathbf{w} \approx q_\pi(s, a)$
- Minimise least squares error between $\hat{q}(s, a, \mathbf{w})$ and $q_\pi(s, a)$ from experience generated using policy π consisting of $\langle \text{state}, \text{action} \rangle, \text{value} \rangle$ pairs

$$\mathcal{D} = \{ \langle (s_1, a_1), v_1^\pi \rangle, \langle (s_2, a_2), v_2^\pi \rangle, \dots, \langle (s_T, a_T), v_T^\pi \rangle \}$$

Least Squares Control

- For policy evaluation, we want to efficiently use all experience
- For control, we also want to improve the policy
- This experience is generated from many policies
- So to evaluate $q_{\pi}(S, A)$ we must learn off-policy
- We use the same idea as Q-learning:

- Use experience generated by old policy

- Consider alternative successor action

$$S_t, A_t, R_{t+1}, S_{t+1} \sim \pi_{old}$$

- Update $\hat{q}(S_t, A_t, \mathbf{w})$ towards value of alternative $A' = \pi_{new}(S_{t+1})$

$$R_{t+1} + \gamma \hat{q}(S_{t+1}, A', \mathbf{w})$$

Linear squares Q-learning

- Consider the following linear Q-learning update

$$\delta = R_{t+1} + \gamma \hat{q}(S_{t+1}, \pi(S_{t+1}), \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})$$
$$\Delta \mathbf{w} = \alpha \delta \mathbf{x}(S_t, A_t)$$

- LSTDQ algorithm: solve for total update = zero

$$0 = \sum_{t=1}^T \alpha (R_{t+1} + \gamma \hat{q}(S_{t+1}, \pi(S_{t+1}), \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \mathbf{x}(S_t, A_t)$$
$$\mathbf{w} = \left(\sum_{t=1}^T \mathbf{x}(S_t, A_t) (\mathbf{x}(S_t, A_t) - \gamma \mathbf{x}(S_{t+1}, \pi(S_{t+1})))^\top \right)^{-1} \sum_{t=1}^T \mathbf{x}(S_t, A_t) R_{t+1}$$

LSPI algorithm

```
function LSPI-TD( $\mathcal{D}, \pi_0$ )  
   $\pi' \leftarrow \pi_0$   
  repeat  
     $\pi \leftarrow \pi'$   
     $Q \leftarrow \mathbf{LSTDQ}(\pi, \mathcal{D})$   
    for all  $s \in \mathcal{S}$  do  
       $\pi'(s) \leftarrow \operatorname{argmax}_{a \in \mathcal{A}} Q(s, a)$   
    end for  
  until ( $\pi \approx \pi'$ )  
  return  $\pi$   
end function
```

LSTDQ for policy evaluation

Policy improvement (greedy)

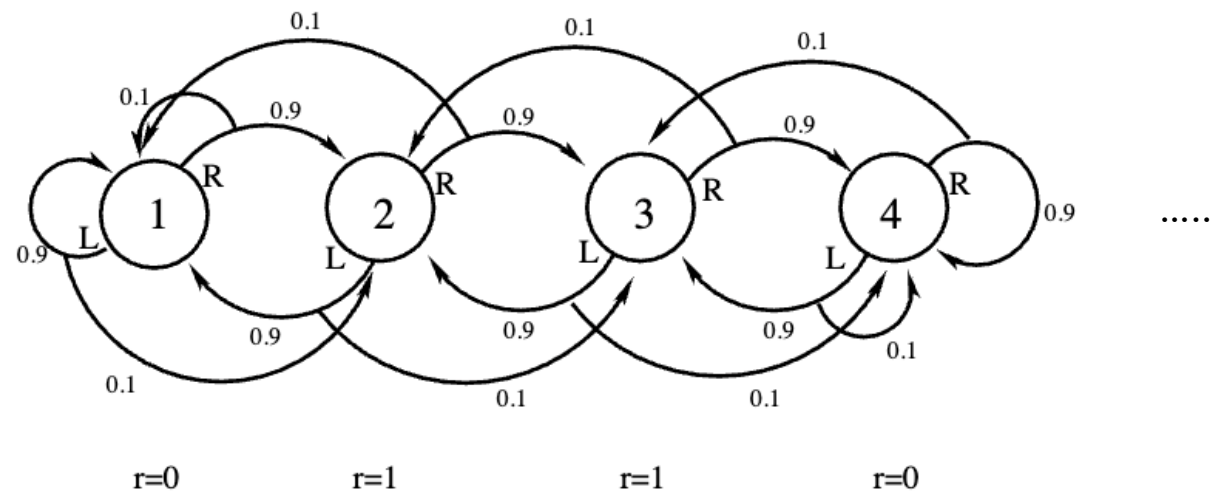
Convergence of Control Algorithms

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	✗
Sarsa	✓	(✓)	✗
Q-learning	✓	✗	✗
LSPI	✓	(✓)	-

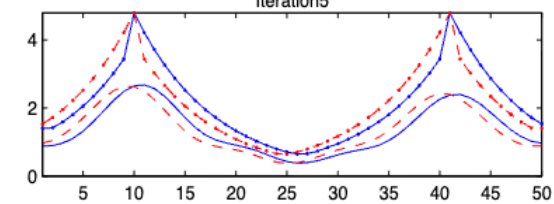
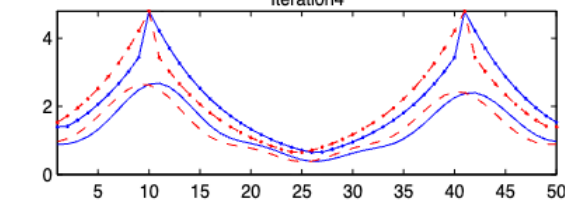
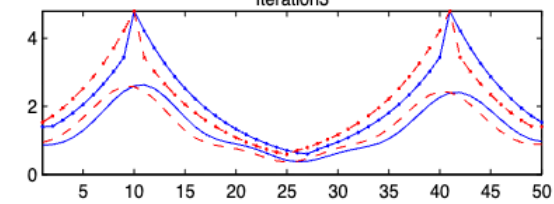
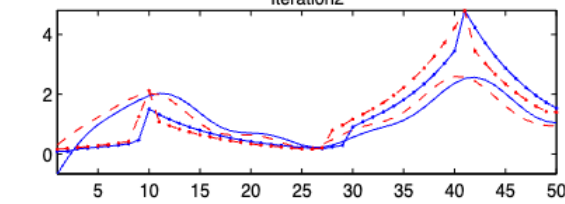
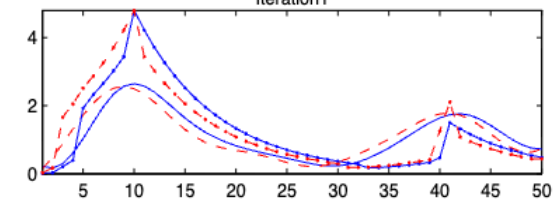
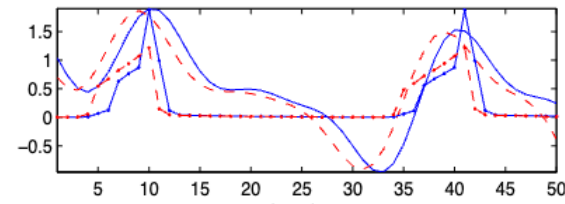
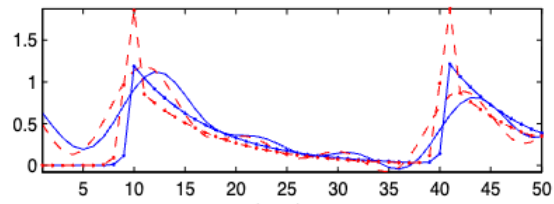
(✓) = chatters around near-optimal value function

Example – chain walk

- Consider the 50-state version of this problem
- Reward +1 in states 10 and 41, 0 elsewhere
- Optimal policy: R (1-9), L (10-25), R (26-41), L(42, 50)
- Features: 10 evenly spaced Gaussians ($\sigma = 4$) for each action
- Experience: 10,000 steps from random walk policy



Example – chain walk



Example – chain walk

